Bread (Sprint 3) — Installation Manual (PDF source)

This markdown file is the source for `INSTALLATION_MANUAL.pdf`. It is also included in the submission zip for convenience.

System requirements

- **OS**: any OS that supports Docker (Linux/macOS/Windows)
- **Docker**: 24+ recommended
- **Docker Compose**: Docker Compose plugin (v2)
- **Browser**: Chrome/Firefox (latest)
- **Disk**: enough space for Docker images + volumes (Postgres/MinIO)
- **Ports**
  - `8080` (default): gateway (frontend + API proxy)
  - `5173` (optional): `frontend-dev` Vite dev server
  - `15432` (optional): Postgres port mapping for local debugging
  - `15672` (optional): RabbitMQ management UI
  - `9001` (optional): MinIO console

Install Docker

If you don't have Docker installed yet:

1. Install Docker for your OS
   - Docker Desktop (recommended on Windows/macOS): follow the official guide: https://docs.docker.com/desktop/
   - Linux: install the Docker Engine + Docker Compose plugin (v2) following the official guide: https://docs.docker.com/engine/ and https://docs.docker.com/compose/
2. Start the Docker service
   - Docker Desktop usually starts automatically after installation.
   - On Linux you may need to start/enable the daemon (exact command depends on your distro).
3. Verify Docker is working

```
docker --version
docker compose version
```

If both commands work, you're ready to run this project with `docker compose up -d`.

Repository contents (what is included)

The submission zip is generated from this repository and includes the files needed to run the system, excluding:

- `.git/`
- `.env` (real secrets)
- `node_modules/`, `.venv/`, `__pycache__/`, build caches, Docker volumes
- `dist/` (derived artefacts)

In practice, the zip will include `docker-compose.yml`, `backend/`, `frontend/`, `nginx/`, `tests/`, and documentation files like `README.md`, `TESTING.md`, and this manual.

Environment setup

1. Create your local environment file:

```
cp .env.example .env
```

2. Adjust variables in `.env` if needed.

Required for local Docker run (dev defaults provided in `.env.example`):

- `TZ`
- `DJANGO_DEBUG`
- `ALLOWED_HOSTS`
- `DJANGO_SECRET_KEY`
- Postgres: `PG_DB`, `PG_USER`, `PG_PASSWORD`
- RabbitMQ: `MQ_USER`, `MQ_PASS`
- MinIO: `S3_ACCESS_KEY`, `S3_SECRET_KEY`, `S3_BUCKET`
- `PUBLIC_MEDIA_URL`
- `GATEWAY_PORT`

Optional (can be blank):

- Gmail OAuth variables (`GMAIL_*`)
- DeepSeek/LLM variables (`DEEPSEEK_*`)

Build & run with Docker Compose

From the project root (same folder as `docker-compose.yml`):

Build images:

```
docker compose build
```

Start the stack:

```
docker compose up -d
```

Check status:

```
docker compose ps
docker compose logs -f
```

Stop:

```
docker compose down
```

To remove data volumes (fresh database / fresh MinIO):

```
docker compose down -v
```

Database setup (migrations / seed)

- The backend container runs migrations at startup.
- If you need to run migrations manually:

```
docker compose exec backend python manage.py migrate
```

How to access the system

- **Main entry (frontend + API gateway)**: `http://localhost:8080` (or `http://localhost:${GATEWAY_PORT}`)
- **Backend API (via gateway)**: `http://localhost:8080/api/...`

Notes:

- The gateway serves the frontend and proxies `/api/` and `/media/` to backend/storage.

Default/demo credentials

For marking and local testing:

- Verification code is fixed to **`1234`** in dev/test mode (no real email required).
- Some flows require `.edu.au` emails (see `tests/integration/README.md` for examples).

Testing instructions

See `TESTING.md` for:

- What tests exist (frontend unit, backend unit, integration helpers)
- How to run each test suite
- What must be tested manually (auth, profile, courses, posts, dashboards, uploads, notifications)

Quick commands:

Frontend unit:

```
cd tests/frontend-unit
npm install
npm test
```

Backend unit:

```
python -m venv .venv
source .venv/bin/activate
pip install -r tests/backend-unit/requirements.txt
cd tests/backend-unit
pytest
```

Troubleshooting

- **Port already in use**: change `GATEWAY_PORT` in `.env` (and restart: `docker compose down && docker compose up -d`)
- **Containers unhealthy**: run `docker compose logs -f` and verify Postgres healthcheck passes
- **Database connection errors**: ensure `postgres` is healthy; check `PG_*` values in `.env`
- **Uploads not accessible**: ensure `PUBLIC_MEDIA_URL` matches gateway URL and that `minio` + `minio-init` are running
- **AI features slow or failing**: leave `DEEPSEEK_API_KEY` empty to disable AI, or increase gateway timeout if enabling
- **Gmail not configured**: leave `GMAIL_*` blank; demo mode should still allow core flows to work